

S

Snooze

T

Time

E

Extension

P

Protocol

Enterprise-Grade Morning Decision Architecture
on AWS Step Functions

Basil Udoudoh | Cloud GTM Lead, Slalom | ATL Cloud Conference 2026

Hey y'all, I'm Basil Udoudoh!

☁ Cloud/Software GTM Lead at Slalom — helping enterprises migrate and modernize on the cloud (AWS, GCP, and Azure)

🏠 17+ years building scalable AWS architectures and leading engineering teams

🚀 Former Head of Technology at Scaleup (seed-stage startup)

⚙ Built production Step Functions workflows — spoiler: we'll see one today

🏈 Keeping up with my lovely wife and kids, tinkering with side tech projects, and rooting for every Atlanta sports team!



Basil Udoudoh

Cloud/Software GTM Lead

Slalom

[linkedin.com/in/basiludoudoh](https://www.linkedin.com/in/basiludoudoh)

How many of you fall into the following categories?

1 What's your role?

Software / Cloud Engineer

DevOps / SRE

Architect

Engineering Manager

2 How deep are you in AWS?

Just exploring

Building things

Production workloads daily

I basically live there

3 Have you used Step Functions?

What's that? 🤔

I've heard of it

Played around with it

Yes, in prod! ✅

Let's get started!



OK, so what is “AWS Step Functions”?

5

“AWS Step Functions” is a fully managed serverless orchestration service that lets you coordinate distributed applications and microservices using visual workflows — called state machines — defined in the Amazon States Language (ASL).



Visual Workflows

Define logic as a state machine in JSON/CDK and see it rendered as a live, interactive diagram in the AWS Console



220+ Integrations

Natively integrates with Lambda, ECS, DynamoDB, SQS, API GW, Bedrock, and hundreds more — no glue code



Built-in Resilience

Retry, Catch, Timeout, and heartbeat are first-class citizens — not bolted-on afterthoughts in your app code

So... why not just chain some Lambdas together?

6



DIY Orchestration



Custom retry/error logic in every function



Zero visibility into what step failed or why



Timeouts are entirely your problem



Parallel execution? DIY thread management



Coordination logic buried in your app code



Paying for Lambda idle time between steps

vs



With Step Functions



Built-in retry with exponential backoff



Visual execution history in the console



Per-state timeouts, handled automatically



Parallel branches are a first-class state



Workflow logic lives in the state machine



Express workflows for high-volume / low-cost

Sounds good! How does Step Functions work?

7

1



Define

Amazon States Language (ASL)

Write your workflow as a JSON state machine — or use CDK/SAM to generate it from TypeScript or Python. States, transitions, retries, and error handling all live here.

2



Deploy

State Machine in AWS

The Step Functions service hosts your state machine. The console renders it visually, tracks every execution, and stores the full history of each run.

3



Execute

& Observe

Trigger via EventBridge, API Gateway, or SDK. Watch each state transition in real-time — with inputs, outputs, and timing for every step.



Task

Do a unit of work — invoke Lambda, call an API, run an ECS task



Choice

Branch logic based on input conditions (the if/else of workflows)



Parallel

Run multiple branches simultaneously and wait for all to finish



Map

Iterate over an array, processing each item (optionally in parallel)



Wait

Pause execution for a duration or until a specific timestamp



Pass

Pass input to output — optionally inject or transform data



Succeed

Terminate the execution successfully — you're done here



Fail

Terminate the execution with a named error and cause message

Standard vs. Express Workflows

9

Standard Workflow	
Execution Duration	Up to 1 year
Execution Model	Exactly-once
Execution Rate	2,000 / sec (default)
History	Full execution history
Pricing	Per state transition
Best For	Order processing, approval flows, long-running critical workflows

vs

Express Workflow	
Execution Duration	Up to 5 minutes
Execution Model	At-least-once
Execution Rate	100,000+ / sec
History	CloudWatch Logs
Pricing	Per execution + duration
Best For	IoT pipelines, streaming, high-volume microservice orchestration



Retry

Automatically re-attempt a failing state. Configure max attempts, initial interval, backoff rate, and which error types to catch.

```
"Retry": [{  
  "ErrorEquals":  
    ["Lambda.ServiceException"],  
  "IntervalSeconds": 2,  
  "MaxAttempts": 3,  
  "BackoffRate": 2  
}]
```



Catch

Catch errors that exhaust retries and route to a fallback state. Think try/catch for your state machine — with full input forwarding.

```
"Catch": [{  
  "ErrorEquals":  
    ["States.ALL"],  
  "Next": "HandleError",  
  "ResultPath": "$.error"  
}]
```



Timeout

Set HeartbeatSeconds and TimeoutSeconds per state. Step Functions will fail the state automatically if limits are exceeded.

```
"TimeoutSeconds": 300,  
"HeartbeatSeconds": 60  
  
// Per-state, not global
```

What is it?

A special Task integration mode where Step Functions pauses execution and generates a unique task token. The workflow is completely suspended — no polling, no compute cost — until an external system calls `SendTaskSuccess` or `SendTaskFailure` with that token.

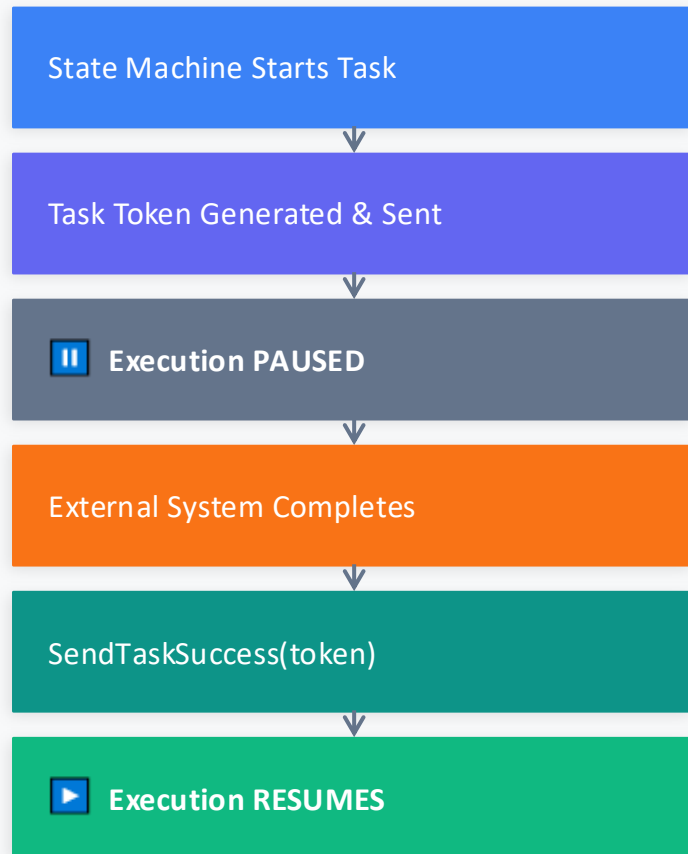
When would I use it?

 Human approval steps (manager reviews a request)

 Waiting for email or SMS confirmation

 Integrating with external async systems

 Literally waiting for a human to get out of bed



Let's Build Something!



Introducing...

S

—

Snooze

T

—

Time

E

—

Extension

P

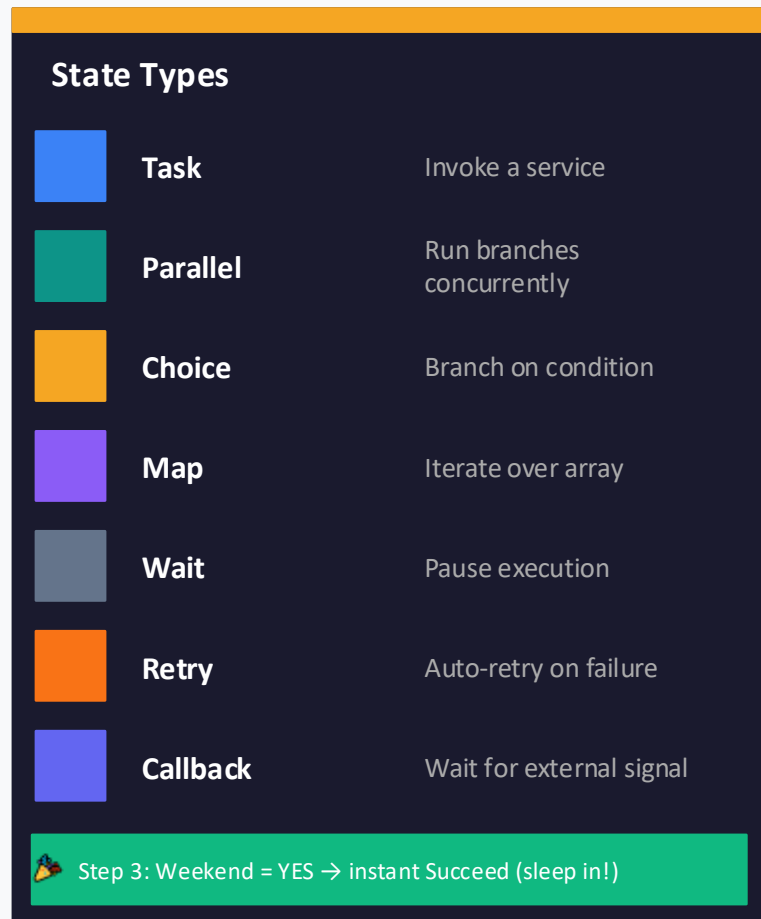
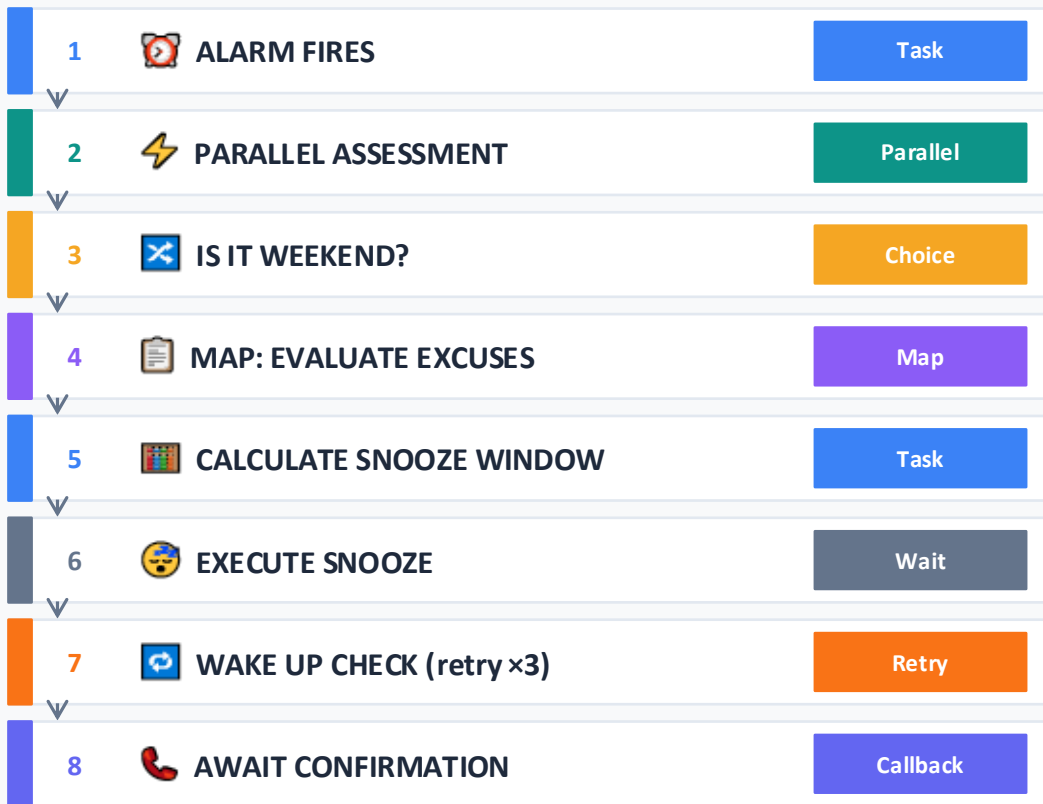
—

Protocol

An enterprise-grade solution to a Monday morning problem

S.T.E.P. Architecture: The Full Workflow

14



SUCCEED: You made it!

FAIL: You're late.

Let's look at the code!



S.T.E.P. — What the Demo Covers

16



Task States

Lambda invocations for alarm check, snooze calculation, and wake-up verification



Choice State

Is it a weekend? Instant Succeed. Otherwise, the gauntlet continues.



Wait State

The literal snooze — execution is paused for the calculated snooze duration



Callback (`waitForTaskToken`)

Workflow pauses until an external signal confirms the human is vertical and conscious



Parallel State

Simultaneous calendar check, weather/outfit complexity rating, and weekend detection



Map State

Iterates over the excuse inventory array, scoring each excuse's viability



Retry + Backoff

Wake-up check retries 3× with exponential backoff (each snooze gets longer)



CDK Deployment

Full TypeScript CDK stack — State Machine, Lambdas, IAM roles, all in one cdk deploy

1



Step Functions transforms coordination code into a visual, auditable state machine — your business logic lives in the workflow, not scattered across Lambda functions

2



Retry, Catch, Timeout, and heartbeat are first-class citizens — you get enterprise-grade resilience without writing a single try/catch in your application code

3



Parallel, Map, and Choice states make complex orchestration trivial — parallel execution, array iteration, and conditional branching all out of the box

4



The `waitForTaskToken` callback pattern unlocks powerful async integrations — pause for human approval, external systems, or (apparently) humans getting out of bed

5



CDK + Step Functions is a killer combo — define your workflow, Lambdas, and IAM in TypeScript and ship the whole thing with one `cdk deploy`



AWS Step Functions Developer Guide

docs.aws.amazon.com/step-functions/



AWS CDK Step Functions Constructs

docs.aws.amazon.com/cdk/api/v2/docs/aws-cdk-lib.aws_stepfunctions-readme.html



Amazon States Language Specification

states-language.net/spec.html



Step Functions Workshop (AWS)

catalog.workshops.aws/stepfunctions/



S.T.E.P. Demo Repo (this talk)

github.com/budoudoh/aws-step-function-demo



Demo Repo →

Thank you!

S

Snooze

T

Time

E

Extension

P

Protocol

Basil Udoudoh | Cloud GTM Lead, Slalom

[linkedin.com/in/basiludoudoh](https://www.linkedin.com/in/basiludoudoh) | github.com/budoudoh



Now go set up a proper wake-up workflow.